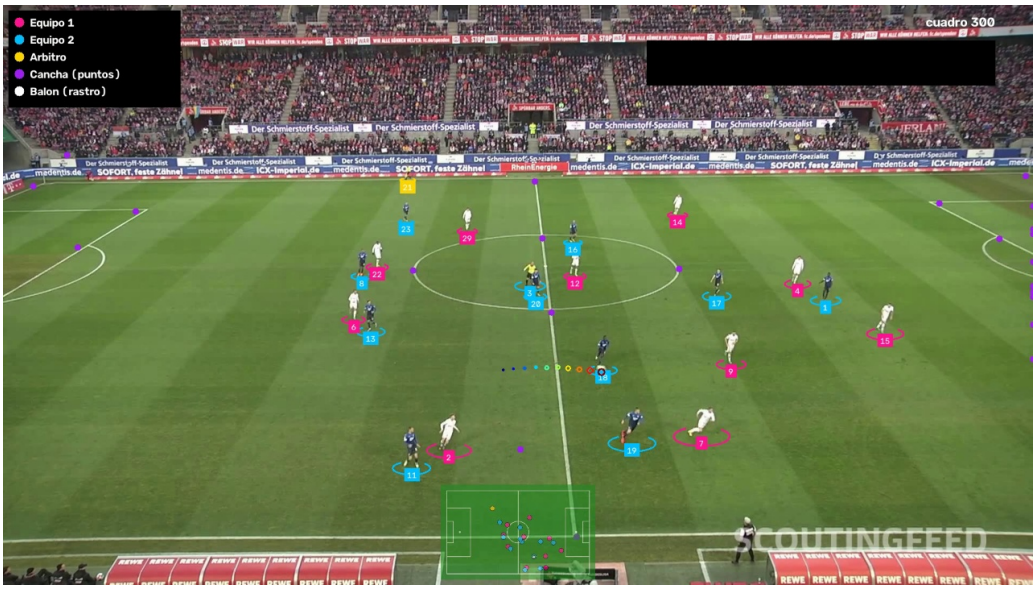
Reporte Técnico
Agentes de Visión por Computadora

Pipeline de análisis deportivo en tiempo real



Destinatarias:	Giselle (Actuaria) · Caroline (Ing. Física)
Proyecto:	Omnimorph Pruebas
Video analizado:	`2e57b9_0.mp4` · 1920×1080 · 25 fps · 30 s
Hardware:	GTX 1660 Ti Max-Q 6 GB · i7-9750H · 24 GB RAM
Estado:	Todos los datos en disco. Sin necesidad de reejecutar.

Generado por el pipeline `scripts/run_together.py` (variante C2) — validado contra los 6 modos originales con IoU medio = 1.0

Índice

1. Contexto del sistema **2**

2. Los 6 agentes **2**
2.1. Agente 1 — PITCH DETECTION . . . 2
2.2. Agente 2 — PLAYER DETECTION . . . 3
2.3. Agente 3 — BALL DETECTION . . . 4
2.4. Agente 4 — PLAYER TRACKING . . . 5
2.5. Agente 5 — TEAM CLASSIFICATION . . . 5
2.6. Agente 6 — HOMOGRAFÍA (ViewTransformer) . . . 6

3. Rendimiento por agente **7**

4. Los datos disponibles **7**
4.1. `objects.parquet` — tabla principal . . . 7
4.2. `frames.parquet` — metadatos por cuadro . . . 8
4.3. `timings.parquet` — tiempos por agente . . . 9
4.4. `muestra_telemetria.json` — snapshot del frame 300 . . . 9

5. Problemas abiertos para el equipo de matemáticas **9**
5.1. P1 — Suavizado de la homografía . . . 10
5.2. P2 — Estimación de velocidad con modelo de movimiento . . . 10
5.3. P3 — Re-identificación de jugadores entre fragmentos de track . . . 10

6. Resumen ejecutivo de los datos disponibles **11**

1. Contexto del sistema

El pipeline analiza video de fútbol y produce, para cada cuadro, las coordenadas reales (en centímetros) de todos los jugadores y del balón sobre la cancha, junto con el equipo de cada jugador y su velocidad aproximada.

Está compuesto por **6 agentes de visión por computadora** que corren en cadena. El sistema fue evaluado sobre 750 cuadros (30 s) del video de muestra. La arquitectura final (`run_together.py`, variante C) procesa todos los agentes en un único proceso GPU, lo que redujo el tiempo total en **43 %** respecto a ejecutarlos en serie (1 316 s vs. 2 328 s).

Sistema de referencia de la cancha (aplica a todos los archivos de datos):

- Cancha: 12 000 cm de largo × 7 000 cm de ancho (120 m × 70 m)
- Eje x: 0 = portería Argentina ⟶ 12 000 = portería Francia
- Eje y: 0 = banda sur ⟶ 7 000 = banda norte
- Unidades: **centímetros**

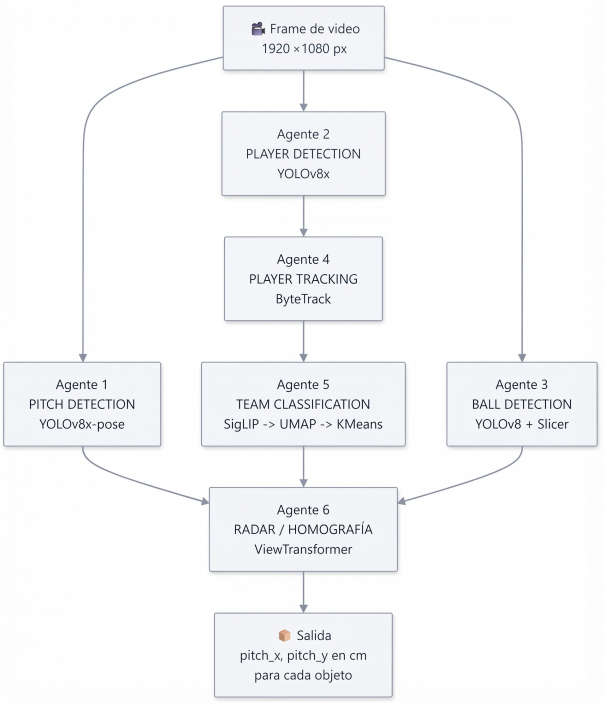


Figura 1: Cadena de los 6 agentes de visión por computadora. Los agentes 1, 2 y 3 corren en paralelo sobre el mismo frame; 4, 5 y 6 dependen de sus salidas.

2. Los 6 agentes

2.1. Agente 1 — PITCH DETECTION

Pregunta que responde: ¿dónde están las líneas y marcas de la cancha en esta imagen?

Modelo: YOLOv8x-pose, entrenado específicamente para detectar landmarks de una cancha de fútbol estándar.

Qué detecta: 32 puntos clave predefinidos — esquinas, manchas de penalti, arcos del círculo central, bordes de las áreas. En promedio **25.9 de los 32** are visibles por cuadro. Los puntos fuera del encuadre tienen confianza ≈ 0 y coordenadas inválidas.

Salida al dataset:

- Serie temporal de matrices de homografía **H** (columna `H[9]` en `frames.parquet`)
- Número de keypoints visibles por cuadro (`n_kp_visible`)
- Coordenadas de los 32 puntos en píxeles (`kp_x`, `kp_y`, `kp_conf`)

Rendimiento medido: 54 ms/frame · 9.6 FPS en modo standalone.

Limitación clave: cuando los keypoints que salen de cuadro cambian entre frames consecutivos, la homografía **H** resultante cambia bruscamente. Esto es la causa principal del ruido en las velocidades calculadas.

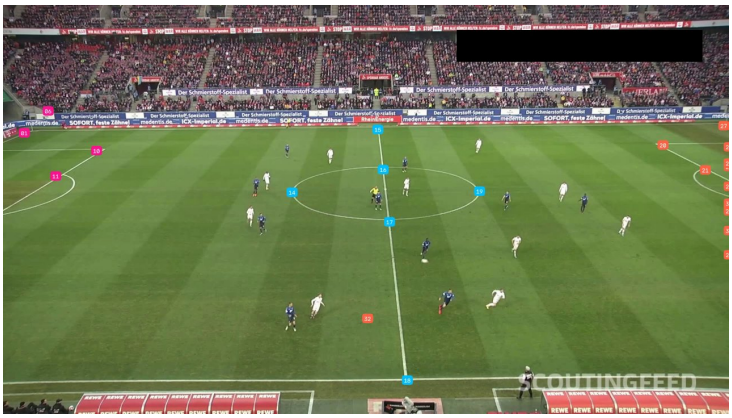


Figura 2: Agente 1 — PITCH DETECTION. Frame 300 (t = 12.0 s). Puntos rosas = zona ARG · Puntos celestes = centro · Puntos naranjas = zona FRA.

2.2. Agente 2 — PLAYER DETECTION

Pregunta que responde: ¿dónde está cada persona en la imagen y de qué tipo es?

Modelo: YOLOv8x a resolución 1280×1280 px (el doble del estándar, para detectar jugadores lejanos).

Clases que detecta:

`class_id`	`class_name`	Descripción
0	`ball`	Balón (detección secundaria, usar Agente 3)
1	`goalkeeper`	Portero
2	`player`	Jugador de campo
3	`referee`	Árbitro o juez de línea

Salida al dataset: caja $[x_1, y_1, x_2, y_2]$ en píxeles, `class_id`, confianza `conf` $\in [0, 1]$. En frame 300: detectó **24 objetos**.

Rendimiento medido: 185 ms/frame · 3.5 FPS standalone.



Figura 3: Agente 2 — PLAYER DETECTION. Frame 300 (t = 12.0 s). 24 objetos detectados: jugadores, porteros, árbitro.

2.3. Agente 3 — BALL DETECTION

Pregunta que responde: ¿dónde está exactamente el balón?

Por qué es especial: el balón mide ≈14 px de diámetro a distancia, demasiado pequeño para un detector estándar.

Estrategia:

1. Divide el frame en cortes solapados de 640×640 px (`InferenceSlicer`)
2. Corre el modelo de balón en cada corte por separado
3. Junta todas las detecciones y aplica NMS (Non-Maximum Suppression)
4. **BallTracker** filtra candidatos eligiendo el más cercano al centroide de las últimas 20 posiciones conocidas:

Listing 1: Lógica del BallTracker — sports/common/ball.py

```
centroid = np.mean(np.concatenate(self.buffer), axis=0)
distances = np.linalg.norm(xy - centroid, axis=1)
index = np.argmin(distances)    # gana el mas cercano al historial
return detections[[index]]
```

Resultado: encontró el balón en **687 de 750 cuadros (91.6 %)**.

Rendimiento medido: 750 ms/frame · 0.95 FPS standalone. **Es el cuello de botella principal: consume el 51 % del tiempo total.**



Figura 4: Agente 3 — BALL DETECTION. Frame 300 (t = 12.0 s). El rastro de color muestra la trayectoria reciente del balón.

2.4. Agente 4 — PLAYER TRACKING

Pregunta que responde: ¿el jugador del frame 301 es el mismo que en el frame 300?

Algoritmo: ByteTrack — empareja detecciones entre frames consecutivos usando IoU (Intersection over Union) de las cajas. Divide detecciones en dos grupos por confianza (alta y baja) y procesa primero las de alta confianza.

Salida al dataset: campo `track_id` (entero) persistente por jugador. En frame 300: 22 de las 24 detecciones recibieron `track_id`.

Rendimiento medido: 6 ms/frame — prácticamente gratuito.

> **Limitación medida — fragmentación de IDs:** En los 30 s del video completo aparecen **39 `track_id` distintos para ≈25 jugadores físicos**. Cada vez que un jugador es ocluido o sale del encuadre, al volver puede recibir un ID nuevo. Esto rompe las series temporales de posición.
>
> **Importante:** `track_id` es un número interno de ByteTrack, **no el dorsal de la camiseta**. No hay correspondencia automática entre ambos.



Figura 5: Agente 4 — PLAYER TRACKING. Frame 300 (t = 12.0 s). Cada jugador lleva su `track_id` numérico.

2.5. Agente 5 — TEAM CLASSIFICATION

Pregunta que responde: ¿este jugador es Argentina (0) o Francia (1)?

Funciona en dos fases:

Fase de entrenamiento (una sola vez, ≈4 min al inicio):

1. Toma 1 de cada 60 frames y recorta la imagen de cada jugador detectado
2. Pasa cada recorte por **SigLIP** (modelo de visión de Google, ≈780 MB, produce vectores de 768 dimensiones que capturan color/textura de la camiseta)
3. **UMAP** reduce de 768D a 3D preservando estructura de vecindad
4. **KMeans($k = 2$)** agrupa los puntos 3D en 2 clusters (un equipo por cluster)

Fase de inferencia (cada frame, 461 ms):

1. Recorta los jugadores del frame actual
2. Pasa por SigLIP → vector 768D
3. Proyecta con UMAP a 3D
4. KMeans predice el cluster → `team_id` = 0 ó 1

Porteros: se asignan por heurística posicional (no por SigLIP):

Listing 2: Heurística de porteros — sports/examples/soccer/main.py

```
team_0_centroid = players_xy[players_team_id == 0].mean(axis=0)
team_1_centroid = players_xy[players_team_id == 1].mean(axis=0)
# gana el equipo cuyo centroide esta mas cerca en pixeles
```

Resultado en frame 300: 10 jugadores Argentina (`team_id`=0), 11 jugadores Francia (`team_id`=1).

Rendimiento: 461 ms/frame — **31 % del tiempo total**.

Limitaciones:

- KMeans y UMAP son no-deterministas. Sin semilla fija, dos corridas del mismo video pueden intercambiar los labels 0 y 1 (corregido en nuestros experimentos con `random_state=42`).
- La heurística de porteros falla si el portero viste un color diferente al resto de su equipo (caso observado en cuadro 100).

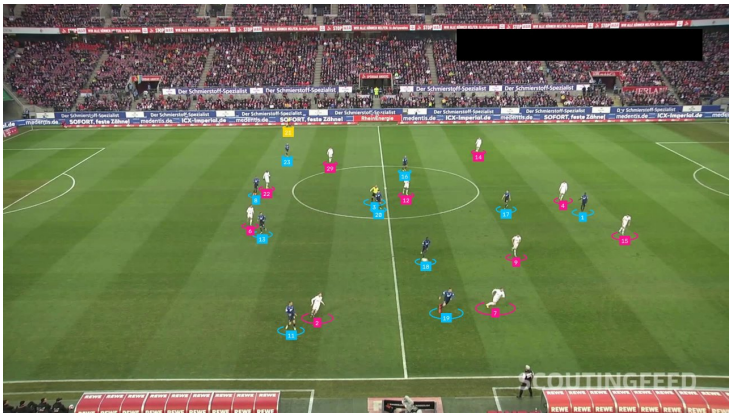


Figura 6: Agente 5 — TEAM CLASSIFICATION. Frame 300 (t = 12.0 s). Rosa = Argentina (`team_id`=0) · Azul = Francia (`team_id`=1).

2.6. Agente 6 — HOMOGRAFÍA (ViewTransformer)

Pregunta que responde: ¿en qué punto real de la cancha está este jugador, en centímetros?

Matemática: toma los puntos del Agente 1 como pares de correspondencia (píxel en imagen ↔ cm en cancha) y resuelve la matriz de homografía $\mathbf{H}$ de 3×3 que mejor los mapea (OpenCV `findHomography` con RANSAC). Con esa $\mathbf{H}$ proyecta cualquier punto de imagen al sistema métrico de cancha mediante transformación perspectiva:

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \mathbf{H} \begin{pmatrix} x_{\text{px}} \\ y_{\text{px}} \\ 1 \end{pmatrix}, \qquad \texttt{pitch_x} = \frac{x'}{w'}, \quad \texttt{pitch_y} = \frac{y'}{w'} \tag{1}$$

Salida al dataset: campos `pitch_x_cm` y `pitch_y_cm` en todas las filas de `objects.parquet`.

Rendimiento: 1 ms/frame — prácticamente gratuito.

Limitación crítica para el modelado: H se recalcula de cero en cada frame usando solo los keypoints visibles en ese frame. **No tiene memoria entre frames.** Si entre dos frames consecutivos un keypoint cambia de posición por detección ruidosa, **H** cambia bruscamente ⇒ las coordenadas `pitch_x, pitch_y` "saltan" ⇒ las velocidades calculadas por diferencia finita dan valores físicamente imposibles.

Ejemplo medido: `track_id`=19 muestra 1 382 cm/s (13.8 m/s = 49 km/h) en frame 300.

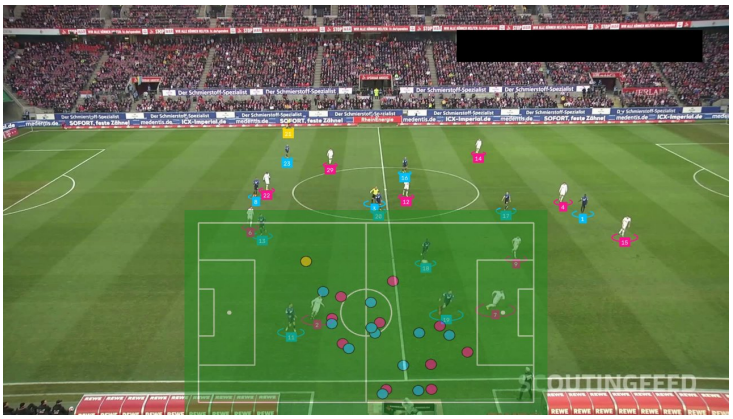


Figura 7: Agente 6 — RADAR. Frame 300 (t = 12.0 s). El minimapa inferior muestra la proyección ortogonal de las posiciones de los jugadores sobre la cancha usando la homografía **H**.

3. Rendimiento por agente

Cuadro 1: Tiempo de cómputo por agente (pipeline C, GPU, media sobre 750 frames)

Agente	Nombre	Media (ms)	p95 (ms)	% total	FPS solo
A3	BALL DETECTION	750	811	**51 %**	0.95
A5	TEAM CLASSIFICATION	461	492	**31 %**	1.12
A2	PLAYER DETECTION	185	190	13 %	3.49
A1	PITCH DETECTION	54	58	4 %	9.58
A4	PLAYER TRACKING	6	10	<1 %	3.29
A6	HOMOGRAFÍA	1	2	<1 %	—
	Total cadena	**≈1 457**			**0.69 FPS**

A3 y A5 consumen el **82 %** del tiempo total y son los candidatos naturales a optimización algorítmica por el equipo de matemáticas.

4. Los datos disponibles

Todos los archivos están en disco. **No es necesario volver a correr ningún modelo** para trabajar con ellos.

4.1. `objects.parquet` — tabla principal

- **Ruta:** `outputs/experiment/C_data/objects.parquet`
- **Dimensiones:** 35 870 filas × 14 columnas
- **Unidad:** una fila = un objeto detectado en un cuadro
- **Cobertura:** 750 cuadros, 30 segundos, a 25 fps

Cuadro 2: Esquema de `objects.parquet`

Columna	Tipo	Descripción
`frame`	int	Cuadro (0–749)
`t`	float	Tiempo en segundos (0.0–29.96 s)
`source`	string	`"det"` = detección bruta A2 · `"track"` = con ID de A4 · `"ball"` = balón de A3
`track_id`	int/null	ID persistente de ByteTrack (null para det y ball)
`class_id`	int	0=balón · 1=portero · 2=jugador · 3=árbitro
`class_name`	string	`"ball"` · `"player"` · etc.
`team`	int/null	0=Argentina · 1=Francia · null=árbitro/balón
`x1, y1`	float	Esquina superior izquierda de la caja (px)
`x2, y2`	float	Esquina inferior derecha de la caja (px)
`conf`	float	Confianza de la detección $\in [0, 1]$
`pitch_x`	float/null	Posición real en cancha, eje largo (cm)
`pitch_y`	float/null	Posición real en cancha, eje ancho (cm)

Cómo leerlo:

Listing 3: Lectura básica de objects.parquet

```
import pyarrow.parquet as pq

df = pq.read_table(
    'outputs/experiment/C_data/objects.parquet'
).to_pandas()

# Solo jugadores con posicion en cancha valida
jugadores = df[
    (df['source'] == 'track') &
    (df['class_name'] == 'player') &
    df['pitch_x'].notna()
]

# Solo el balon
balon = df[df['source'] == 'ball']

# Un solo frame
frame_300 = df[df['frame'] == 300]
```

4.2. `frames.parquet` — metadatos por cuadro

- **Ruta:** `outputs/experiment/C_data/frames.parquet`
- **Dimensiones:** 750 filas × 13 columnas
- **Unidad:** una fila = un cuadro completo

Cuadro 3: Esquema de `frames.parquet`

Columna	Tipo	Descripción
`frame`	int	Cuadro (0–749)
`t`	float	Tiempo en segundos
`n_det`	int	Objetos detectados por A2
`n_track`	int	Objetos con `track_id` de A4
`ball_found`	bool	A3 encontró el balón
`homography_valid`	bool	A6 calculó **H** con éxito
`homography_error`	str/null	Mensaje de error si **H** falló
`n_kp_visible`	int	Keypoints de cancha visibles (calidad de **H**)
`kp_x`	list[float×32]	Coordenadas x en px de los 32 keypoints
`kp_y`	list[float×32]	Coordenadas y en px de los 32 keypoints
`kp_conf`	list[float×32]	Confianza de cada keypoint
`H`	list[float×9]	Matriz **H** 3×3 aplanada (fila por fila)

Para P1 (suavizado de homografía): la columna `H` es la secuencia temporal de 750 matrices 3×3. La columna `n_kp_visible` es el indicador de calidad de cada estimación. Juntas son el input para proponer un filtro temporal sobre **H**.

4.3. `timings.parquet` — tiempos por agente

- **Ruta:** `outputs/experiment/C_data/timings.parquet`
- **Dimensiones:** 750 filas × 9 columnas
- **Unidad:** una fila = un cuadro; cada columna = ms de un agente

Columnas: `frame`, `decode_ms`, `pitch_ms`, `player_ms`, `ball_ms`, `tracking_ms`, `teams_ms`, `homography_ms`.

4.4. `muestra_telemetria.json` — snapshot del frame 300

- **Ruta:** `muestra_telemetria.json` (raíz del proyecto)
- **Contenido:** estado completo del partido en el segundo 12.0
- **Propósito:** prototipado rápido sin necesidad de leer los parquet

Cuadro 4: Situación táctica en frame 300 (t = 12.0 s) extraída del JSON

Quién	team	pitch_x (cm)	pitch_y (cm)	speed (cm/s)
Balón	—	7 335	5 592	—
FRA track_id 18	1	7 337	5 565	600 (junto al balón)
ARG track_id 2	0	6 722	6 510	187 (presionando)
ARG track_id 15	0	9 637	5 049	490 (dentro del área FRA)
ARG track_id 7	0	8 341	6 498	817
FRA track_id 19	1	7 877	6 541	1 383 ← ruido H
Árbitro (tid 21)	null	3 858	1 505	78 (casi quieto)

5. Problemas abiertos para el equipo de matemáticas

Los tres problemas siguientes están **listos para modelar** con los datos ya en disco, sin tocar los modelos de visión.

5.1. P1 — Suavizado de la homografía

Agente causante: A6 recalcula **H** cada frame sin memoria.

Síntoma: cuando un keypoint de cancha entra o sale del encuadre entre dos frames consecutivos, **H** cambia bruscamente y las posiciones (`pitch_x`, `pitch_y`) "saltan" decenas de metros.

Input disponible:

- Serie temporal de 750 matrices **H** (columna `H[9]` en `frames.parquet`) — frecuencia 25 Hz
- Calidad de cada estimación: `n_kp_visible` (0–32)
- Indicador booleano de validez: `homography_valid`

Pregunta para el equipo: ¿qué filtro temporal sobre **H** (o directamente sobre las coordenadas proyectadas) reduce los saltos preservando los cambios reales de la cámara?

5.2. P2 — Estimación de velocidad con modelo de movimiento

Agente causante: el script `build_telemetry_snapshot.py` calcula velocidades por diferencia finita simple entre frames ±5, sin modelo de movimiento ni manejo de observaciones faltantes.

Síntoma: velocidades físicamente imposibles (49 km/h para un jugador parado) cuando **H** es ruidosa.

Input disponible:

- Serie temporal $\{(t_k, x_k, y_k)\}$ por `track_id` en `objects.parquet` — frecuencia nominal 25 Hz
- Los frames con `homography_valid = False` son observaciones faltantes (en este video: 0 de 750, pero sí ocurre en otros clips)
- La confianza de cada posición está en la columna `conf`

Pregunta para el equipo: ¿cómo estimar velocidad instantánea robusta a los saltos de homografía y a los frames faltantes?

5.3. P3 — Re-identificación de jugadores entre fragmentos de track

Agente causante: A4 (ByteTrack) asigna `track_id` nuevo al jugador que reaparece después de una oclusión.

Síntoma: 39 `track_id` distintos para ≈25 jugadores físicos en 30 s de video.

Input disponible:

- Series temporales incompletas de posición por `track_id` en `objects.parquet`
- Los IDs con series cortas (< 20 frames) son probablemente fragmentos de jugadores que ya tienen otro ID
- `team_id` está disponible para cada fragmento (restricción: dos fragmentos del mismo jugador deben tener el mismo equipo)

Pregunta para el equipo: ¿cómo asignar fragmentos de trayectoria al mismo jugador usando su posición, equipo y velocidad?

6. Resumen ejecutivo de los datos disponibles

Cuadro 5: Inventario de datos listos para modelar

Archivo	Filas	Cols	Para qué sirve
`objects.parquet`	35 870	14	Posiciones, equipos, confianzas · P2, P3
`frames.parquet`	750	13	Matrices H, calidad de keypoints · P1
`timings.parquet`	750	9	ms por agente · análisis de rendimiento
`muestra_telemetria.json`	1 frame	—	Prototipado inmediato · P1, P2, P3
`contexto_partido.json`	32 jug.	—	Diccionario dorsal → nombre

El dato es real, medido sobre un partido de la Copa del Mundo 2022, y está en disco. No hace falta correr ningún modelo para empezar a modelar.